

Prime number, valid case: int	Test Case	Test Data	Expected Result
Valid case : Input take only integer	1.) prime num 2.) int>1 but non-prime 3.) int <= 1	1.) 2, 7, 5, 997 2.) 4, 9, 25 3.) 1, 0, -2	1.) True 2.) False 3.) False
Invalid case	4.) float 5.) char 6.) string 7.) Null	4.) 0.2, -0.04, 0.03 5.) d, h, i, @, & 6.) Pla, Mind 7.) Null	Reject

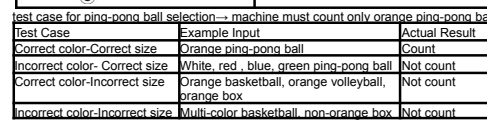
White Box Testing → test only valid case if they go to the correct path or not

Test case → program path	Test Data	Expected
when $i = 0$	0	'Yes'
when $i = 1$	1	'No'

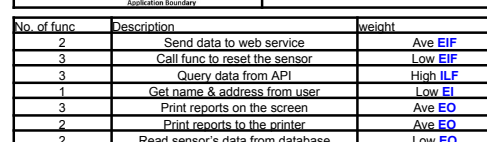
Ideal: ગુરુત્વાત્મક પથ = ગુરુત્વગ્રસ્ત output

If #path != #output

ex.) has only 2 path instead of 3
→ sth wrong with ur code



Function type 1. Internal Logical File (ILF) → data stored in sys 2. External Interface File (EIF) → data stored in another sys 3. External Input (EI) → data entering sys e.g., click button, fill form 4. External Output (EO) → data leaving sys with processing e.g., display, save data to storage 5. External Inquiry (EQ) → simply query (no processing)



Example Calculate sys total cost from given subsys

Functions	Estimated LOC	LOC/P-month	\$/LOC	Cost	Effort

Line of Code (LOC) : Physical LOC = count line of program src except comment &

1. main()	Physical LOC = 4 (1,2,4,6)
2. {	Logical LOC = 2 (1,4)
3. // Print Hello	Comment = 1 (3)
4. printf("Hello\n");	SW productivity = tell quantity not quality
5. }	Productivity measures by size (LOC) or functions (not func)
6. }	
1. for (i = 0; i < 100; i++) printf("hello");	Physical LOC = 1
2. //How many lines of code is this ?	Logical LOC = 2 (loop=1, print=1→sum)

<pre> VerPlanVal = (VerPlanVal == false) ? true : false; } //Verbose Code if (bePlanVar == false) { bePlanVar = true; } else { bePlanVar = false; } </pre>	Cognitive LOC = 1 (2,4) Comment = 1 Physical LOC = 5 (2,3,4,5) Logical LOC = 4 (2,4) Comment = 1 The higher verbose, the higher product The lower lv language, "
--	---

```

graph LR
    A[Number of lines of code reused or generated] -- Based on --> B[Reuse model]
    B -- Used for --> C[Effort to integrate reusable components automatically generated code]

```

$PM = (NAP \times (1 - \%reuse/100)) / PROD$ o NAP is the effort in person-months o NAP or NOP is the number of application points or object points o PROD is the productivity (NOP / month)	Assume 30% reuse, CMMI Assessment nominal LV. $\rightarrow PROD = 13$ (from table Obj) Point Productivity, 2 simple screen & 3 medium screens (ถ้าใช้แล้วมาคูณด้วยจำนวน table Object Point: Complexity LV and Weighting) $\rightarrow object = 2 \times 1 + 3 \times 2 = 8 = NOP$ $PM = (8 / (1 - 0.3)) / 13 = 0.43$ p-months
---	---

Early Design model: 7 factors, use when requirements are available but design has

→ Depending on novelty of the project, development flexibility, risk management approaches and the process maturity

Multipliers (M) reflect the capability of the developers, the non-functional requirements, the familiarity with the development platform, etc. RCPX - product reliability and complexity, RUSE - the reuse required, PPDF - platform difficulty, PREX - personnel experience, PERS - personnel capability, PSCD - required schedule, PFCL - the personnel

account black-box code that is reused without change and/or code that has to be redesigned or integrated with other code. 2 Versions: 1. Black-box reuse where code is reused as-is. Effort estimation (PM) is computed as $PM = \text{White-box reuse where code is modified} + A$. A is estimation equivalent to the number of lines of new source code is computed. And then adjusts the size estimation for new code.		
Black box For generated code PM = (ASLOC * AT1/10)/ATPROD n ASLOC is the number of lines of generated code	For Generated code example: PM = (20,000 * 2.0) / 100 = 400	

- costs of changing the reused code, the costs of understanding how to integrate the code and the costs of reuse decision making.
- AAU is the sum of three components (AAF+SU+AA)
 - AAU (Understanding Component) ranges from 0 to 15 for complex to 10 for well-written, object-oriented code
 - AA (Assessment Component) varies from 0 to 8 depending on the amount of analysis effort required
 - AAF (Adaptation Adjustment Factor)(Adaptive Component) :

$CM = 20 : 0$
 $IM = 20 : 0$
 $AAF = (0.4 \cdot 0)$
 $(0.3 \cdot 0.2) +$
 $(0.3 \cdot 0.2) = 0$
 $AAU = 0.28$
 $A = 3 \cdot 33.28$

affecting the software, Personnel attr: Skills and experience of the development team
Project attr: Characteristics of the project (e.g., complexity, timeline)

Effort Adjustment Factor (referring from 1)
The Effort Adjustment Factor in the effort

Cost Drivers	Ratings					
	Very Low	Low	Normal	High	Very High	Extra High
Product attributes						
Required working velocity	0.75	0.88	1.00	1.15	1.40	
Size of application database		0.84	1.00	1.08	1.16	
Complexity of the product		0.70	0.85	1.00	1.15	1.65
Hardware attributes						
Number of performance constraints			1.00	1.11	1.30	1.65
Memory constraints			1.00	1.08	1.21	1.56
Velocity of the virtual machine environment		0.87	1.00	1.15	1.30	

	Development	Testing 2, Validation testing
	Project Scheduling Process : Identify activity (sf requirement) → iden act dependencies → Estimate resources for act → Allocate ppl to act → create proj chart eg., gantt	
	Estimate Task Duration : OD (Optimistic dur or fastest time), PD(Pessimistic dur or max time) ED (Expected or Avg) → Duration(D) = round_up(“(OD + 4 * PD + ED)” / 6)	
Activity	DevOps Process : Process = CI/CD pipeline: Plan → Code → Build or Integrate → Test → Release → Deploy → Operate or Monitor. It is a continuous cycle to deliver software quickly and reliably. DevOps process is characterized by a “Continuous Pipeline”	
Sector		

- **Continuous Delivery:** Ensures that code is always in a deployable state to a testing or staging environment. → Tools: Ansible, Docker, and Spinnaker.
- **Continuous Deployment:** Automatically deploying production code changes directly into the production environment without manual intervention. → Tools: Kubernetes, AWS CodePipeline, and Google Cloud Build.
- **Continuous Monitoring:** Real-time tracking of application performance and infrastructure health. → Tools: Prometheus, Grafana, Zabbix, ELK stack, and Datadog.

- Runs the same in all environments (dev, test, production)
- Lightweight (faster than virtual machines) → Containers (like Docker) are lightweight; they sit on a Docker Engine and share the Host OS kernel, meaning they don't need a Guest OS and work much faster than VMs.
- Orchestration: Because modern apps use many containers, tools like Kubernetes, Docker Swarm, or OpenShift are used to manage, scale, and schedule them.
- Easy to deploy and scale

Scenario (Chat)

Scenario 1: A university is developing an IC Library system. Students complain that the system is slow to solve. During a meeting, the system officer is told that the

3. **Containerization** Use Docker: 1. Package app + dependencies into a container 2. Run the same in dev, test, and production

Solves: "Works on my machine" problem and Easier deployment and scaling

4. **DevOps Solution Summary** DevOps improves the system by: 1. Speed → automatic deployment 2. Quality → continuous testing 3. Stability → consistent environment with containers 4. Collaboration → dev & ops work together

"DevOps uses CI/CD, automation tools, and containerization to deliver software faster, reduce errors, and ensure consistent deployment across environments."

DevOps Process DevOps pipeline → set of practice that keeps process organized and focused (Consistent DevOps (SCM)) → systematically manage, organize & control:

Version Control: tracking, managing changes to software code over time, allow dev to collaborate, reverse mistakes. **Release Management:** managing, planning, scheduling, and controlling a software build through diff stages & environments; testing and deploying software releases **Work-item Tracking:** managing tasks, bugs, and requests throughout the dev lifecycle. **Traceability:** trace requirements, changes, & test results to ensure all components are accounted for. **Baselining:** a stable, verified version of a configuration

- collaborating? **Automation:** is the boring work handled by code / **Lean:** A/E updates small and fast? **Measurement (KPIs):** Are we tracking speed and quality (DORA metrics)? **Sharing:** Is knowledge open to everyone?
- DevOps toolchain:** collection of software tools that automate the tasks of building, testing, and delivering software. It creates an "assembly line" that connects developers and IT operations. **The Continuous Delivery pipeline:** **Develop:** Plans, tests, and builds code. **Package:** creates a software source code. **Deliver:** automatically building and testing code (CI/CD). **Deploy:** Packaging apps into containers and managing cloud resources. **Detect:** Monitoring infrastructure, alerting on changes, ensuring availability. **WPA 2.0 Automation:** Reduces manual